Runaway Electron Loss Distributions on 3D Wall Structures

Internship Report

Michail Marios Skyllas - 1747452

November 28, 2022

Contents

List of Figures **2**

1 Introduction **1**

2 Structure **4**
2.1 The JOREK Simulation 4
2.2 Particle Tracing 5
2.3 Triangle Search Optimization - Octree 7

3 Results **9**
3.1 Octree Convergence 9
3.2 Number of Markers Scan 10
3.3 Pitch Angle Scan 13
3.4 Energy Scan 15
3.5 Preliminary Heat Loads Results 18

4 Summary & Future Work **20**

List of Figures

1 Evolution of post-disruption plasma current in ITER when runaway electrons are not mitigated. Evolution of the runaway electron current is indicated with the orange and red curves. The blue and green curves depict the evolution of the total current. At the plateau phase the runaways can carry a very large fraction of the total initial current. The two scenarios (blue/orange and red/green) only differ in internal inductance. Figure source: [1] 1

2 The evolution of the total runaway electron current during the simulated disruption. 4

3 The markers' density and positions. The density given as an input to the aforementioned script is converted into particles (markers) at positions that approximate the original density distribution. 5

4 The Octree decomposition scheme, adapted from [17] 7

5 The amount of CPU time required to simulate the markers, each time for a different maximum depth of the octree structure. The blue curve is the asymptotic exponential fit used on the run data. 10

6 The (R,Z) plot on the left displays which markers have been registered as lost or confined. The blue curve is used in postprocessing to measure how far along the wall losses span. The (ϕ, θ) plot shows the surface density of the hits along the torus. This example is the result of a simulation of 10,000 markers with a kinetic energy of 100 MeV each. 11

7 The spatial distribution of the hits when projected along the curve of the wetted area of the wall, for a different number of markers all of which had a kinetic energy of 100 MeV. 12

8 The fraction of lost particles during a complete simulation for a runtime of 0.01, 0.1 and 1 ms, for a default energy of 100 MeV. 13

9 The normalized peak particle load and wetted area for an increasing number of markers. 13

10 The number of lost particles for different values of the pitch, for 10000 runaway electrons of 20 and 100 MeV. Each color represents the simulation time used in each case, spanning 2 orders of magnitude. 14

11 The spatial distribution of the hits along the curve of the wetted area of the wall, for a varying value of the pitch. 15

12 The total number of lost particles as a function of the energy of the markers for varying simulation times. The total number of markers was 10,000. . . . 15

13 The spatial distribution of the hits along the curve of the wetted area of the wall, for a varying value of the kinetic energy of the markers. 17

14 The 3D representation of the model of the wall with the energy load from the losses is being indicated by the coloration of the cluster of triangles. Note that the wall model used for this demonstration does not exactly reflect the ITER wall structure such that our results do not provide quantitative predictions at this stage. 18

15 The (R,Z) plot shows where the particles were deposited and the (ϕ, θ) plot depicts the particles deposition around the torus. 19

16 The heat flux distribution projected onto the blue curve from figure 15, across the torus. 19

Abstract

In tokamak plasmas, disruption scenarios are the birthplace of runaway electrons, perhaps the most dangerous species of particles that can exist within a fusion experiment. This danger stems from the fact that they can form an energetic beam that has the potential to cause severe damage on the first wall materials.. In this report we set out to build a tool with which we will be able to acquire the spatial distribution of the runaways on the first wall, which can then be used to assess the potential damage. The primary components of the tool are the Möller - Trumbore algorithm for resolving when particles intersect the wall elements and the octree structure algorithm for the segmentation of 3D space into octants that speeds-up the wall collision detection process. These were integrated into a relativistic guiding center tracer that is built within the MHD code JOREK, which is frequently used to study disruptions and model runaway electrons. The new algorithm was then used to postprocess an existing simulation of an ITER disruption, where runaway electron beam dynamics were modelled as a fluid. The octree optimization was found to be effective and the results converged with increasing number of markers with 10^5 being sufficient to estimate the peak loads on the wall. The distribution of the wall loads was found to weakly depend on pitch but more significant dependence on energy was seen above at energies above 30 MeV. The preliminary assessment gave peak loads for which the 3D structure of the wall had significant contribution, but the runaway electron energy in this case was a conservative estimate. While this project built the tools for determining RE loss patterns, demonstrated convergence and applications to realistic scenarios, concrete predictive simulations for ITER are left to future work due to the limited time.

1 Introduction

During a disruption, the electric field dramatically increases and causes the supra-thermal electrons in the plasma to undergo "freefall" acceleration, usually rendering them relativistic. This effect is a consequence of the friction force decreasing with increasing electron velocity [1]. The name "runaway" electrons stems from the corresponding threshold, i.e., when the accelerating electric force is equal to the friction force. This beam of relativistic electrons is perilous for the first wall of the vacuum vessel upon establishing contact with it. In fact their impact is so detrimental that reactor-scale tokamaks cannot allow a sizable runaway electron beam to form [2].

At the beginning of a disruption, the plasma undergoes a thermal quench, which is a drastic loss of the plasma's kinetic energy, and a current quench, which is defined as the loss of the current carried by the plasma. However, this current quench turns into a current plateau if a runaway electron beam is formed and the whole plasma current is eventually carried by runaway electrons [1, 3] (see figure 1).

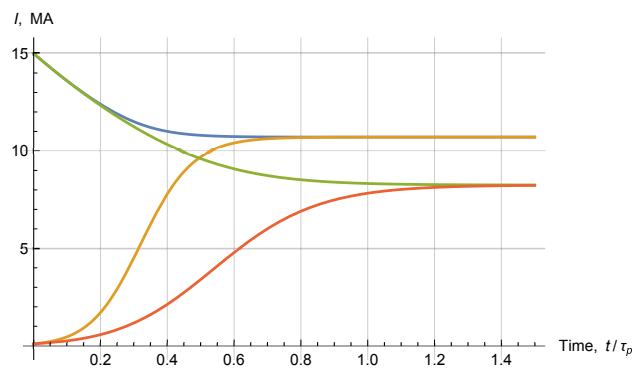


Figure 1: Evolution of post-disruption plasma current in ITER when runaway electrons are not mitigated. Evolution of the runaway electron current is indicated with the orange and red curves. The blue and green curves depict the evolution of the total current. At the plateau phase the runaways can carry a very large fraction of the total initial current. The two scenarios (blue/orange and red/green) only differ in internal inductance. Figure source: [1]

A plasma may disrupt due to the influence of two independently acting factors. The first one is the influx of impurities in the plasma, despite their purposeful injection (or lack thereof), which are known to be strong line radiation emitters. The other factor is global MHD events which facilitate heat transport by means of stochastization of the magnetic field lines [1]. The aforementioned independence implies that one may lead to the other and vice versa.

The thermal quench is the one which induces runaway production. Compared to the timescale at which the toroidal current changes significantly (the inductive timescale), the steep plasma temperature drop takes place in a faster scale. Then, considering the resistivity of the plasma rises with the decreasing temperature, the electric field is required to increase since the current needs to be conserved. This amplified electric field makes it possible for electrons to be converted into runaways.

The runaway electron generation mechanisms can be divided into primary and secondary

mechanisms. The primary mechanisms are the ones that do not require a pre-existing population of runaway electrons to generate new ones, in other words no "seed" is needed. The first primary mechanism is due to diffusion in momentum space, from the tail of the Maxwellian to the runaway regime and is called Dreicer generation bearing the name of its discoverer [4, 5].

The second primary mechanism is called the "hot - tail" generation. This revolves around the rapid cooling of the plasma, which as was mentioned before, is a characteristic trait of the thermal quench during a disruption [6, 7]. The premise of this generation mechanism is that, when the thermal quench timescale is considerably smaller than the timescale required for equilibration by means of collisions, the electrons that once constituted the high energy part of the plasma bulk (before the temperature drop), now form a protracted hot tail at the new temperature after the drop. The presence of an electric field will accelerate those electrons that are included in the runaway region in momentum space. The hot-tail mechanism is projected to be more relevant for ITER rather than the Dreicer generation [8]. Runaways can also be born via Compton scattering and beta decay of tritium, both of which will be also significant mechanisms in ITER.

Secondary generation of runaways involves the collision of pre-existing runaways with thermal electrons, during which, sufficient momentum is transferred so that the latter is converted into a runaway while the former remains a runaway post-collision [9, 10]. Those are the so called "knock-on/close/large-angle" collisions in contrast to small-angle scattering that is more typical in fusion plasmas. This implies that as the number of runaways increases, so does the number of knock-on collisions, which results in the birth of more runaways. Therefore this process will lead to an exponential growth of the runaway electron population, or in other words a runaway electron avalanche.

Having established the way runaways are created and why their existence is unwanted in all present and future tokamaks, it is clear that both prevention and mitigation techniques are needed for the experiments. However, those require substantial information around runaways. One of the most crucial bits of knowledge is the exact distribution pattern of the runaway electron beam when it impacts the wall and what kind of energy loads can we expect. In literature there is not sufficient conclusive evidence or a fully developed and proven theory on the matter. A certain piece of work by Papp was presented at the 16th Congress on Plasma Physics, in which it is shown that in the presence of resonant magnetic perturbations helps localise the deposition of the runaway beam [11] but this study did not include the magnetic field stochasticity inherently present in disruption.

In this work, we study the runaway electron distribution on the first wall elements when those particles are lost during a disruption. This study is comprised of the following parts.

1. First, we develop a fast and efficient algorithm which detects when a runaway electron intersects a wall element. This will be implemented into the built-in particle tracer found within the MHD code JOREK.

2. Second, we study how the wall distribution is affected by the energy and the pitch of the runaways, as their values are not yet fully known.

3. Finally we assess whether the runaway electrons generate hot spots on the wall and whether those originate from the wall structure.

2 Structure

2.1 The JOREK Simulation

The first step towards the realization of the project is to simulate an ITER disruption scenario to achieve the birth of runaways and the corresponding runaway plateau. The MHD group studying runaway electrons at IPP provided said simulations [12]. The simulations were carried out using JOREK, which is a non-linear extended MHD code which has proven to be extremely versatile in regards to its usability [13]. It can simulate realistic tokamak X - point plasmas as well as large scale instabilities. It also includes a fluid model for runaway electrons [14], that can be used to study the beam evolution and which was used in these simulations.

The output of interest from this particular simulation is the runaway current as can be seen in figure 2. It is apparent that the plateau region is reproduced and this ends in a rapid loss of runaway electrons as the beam drifts to the wall and 3D MHD instabilities develop. It is this loss-phase that we aim to simulate with the particle tracing module found in JOREK.

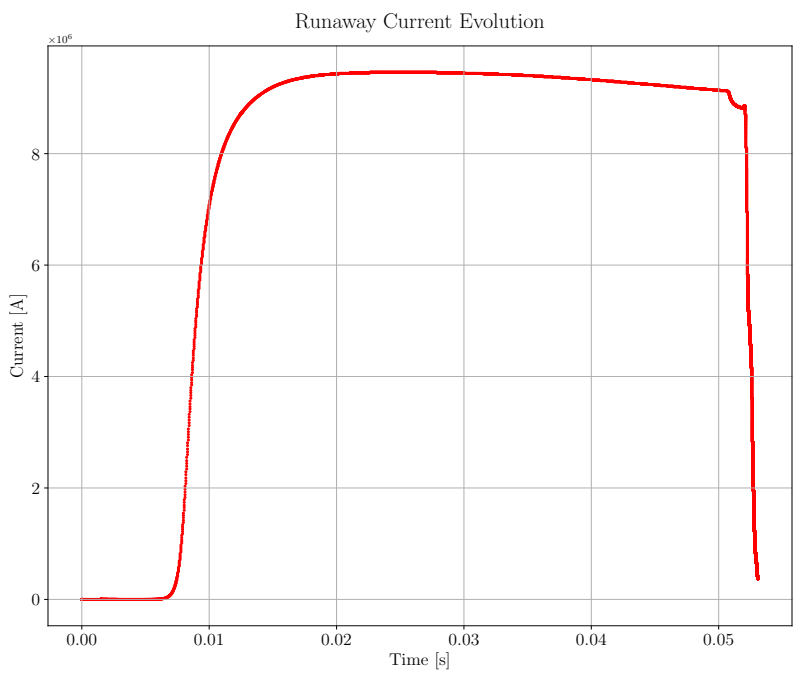


Figure 2: The evolution of the total runaway electron current during the simulated disruption.

In order to consume the least amount of resources while simulating the runaways i.e. register losses of particles as soon as possible, it seems as if the appropriate time to generate the markers (particles) is at the end of the plateau region just before the steep drop of the current. This corresponds to approximately 0.052 seconds.

2.2 Particle Tracing

With the necessary input data to study runaway electron losses at hand, the next step would be to generate the markers for particle tracing. Markers are generated by sampling the runaway electron density that was obtained in the fluid simulation (figure 3). The number of markers and their kinetic energy and pitch are free parameters which are varied to see how sensitive the results are to these.

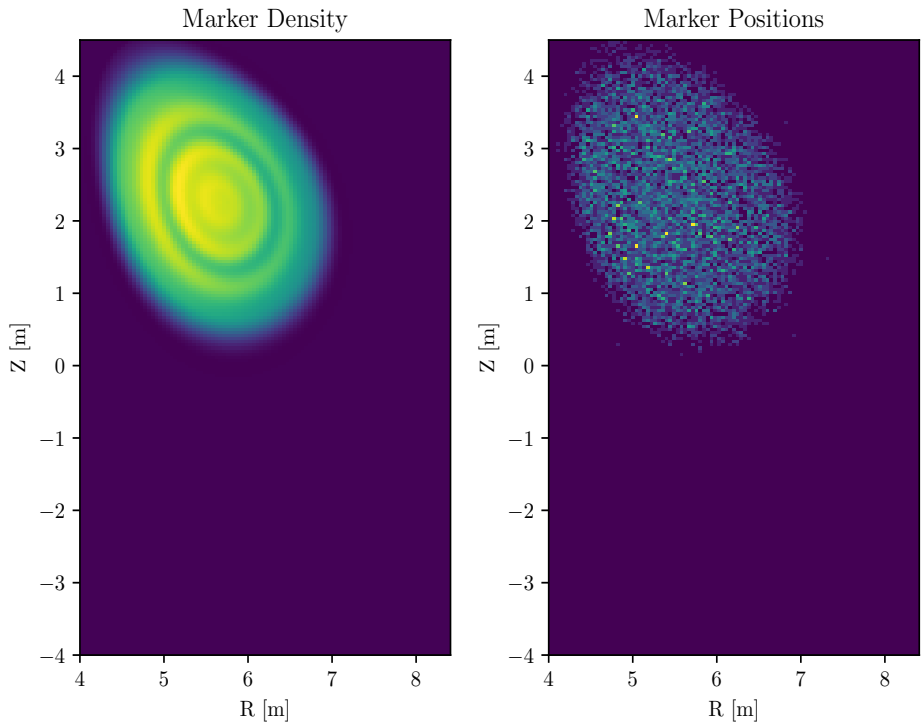


Figure 3: The markers' density and positions. The density given as an input to the aforementioned script is converted into particles (markers) at positions that approximate the original density distribution.

In the simulations, the markers are traced using the built-in particle module that solves their relativistic guiding center orbits using the time-evolving magnetic field data from the fluid simulation as an input. Note that this is a "post-processing" tool and there is no feedback from the markers to the fluid equations. The markers are traced for a given time or until they are lost.

This is the foundation upon which the project is built. Since the goal of this project is to acquire information about the spatial distribution of the runaways, one has to implement an algorithm, that is capable of determining whether a particle has come in contact with the wall. The wall itself is modeled as a triangular mesh which consists of 470,000 triangles. The concept of the algorithm can be reduced to the following sequence of actions for every timestep and for every particle:

1. Provide the particle's current and previous position in space.

2. Check whether the line segment, formed by the aforementioned points in space, intersects any of the triangles of the mesh modelling the wall.

The elementary problem of determining whether a line segment and a triangle intersect in 3D space is called ray tracing and it is not new at all. In fact, it is a technique ubiquitous in video game light physics design (hence the name ray tracing) as well as the development of graphic tools. There have been numerous solutions to this problem, however, it is necessary to apply some restrictions. We need a fast, minimum storage algorithm and it would have to return the position of the intersection on the triangle at hand.

In that regard, the Möller - Trumbore algorithm [15] seems to fit our qualifications while retaining a level of simplicity.

The algorithm revolves around defining a ray $R(t)$ with an origin O and a direction D, with a scalar t as:

$$\mathbf{R}(\mathbf{t}) = \mathbf{O} + t\mathbf{D} \tag{2.1}$$

Then, for the triangle one only needs the three vertices $\mathbf{V_0}, \mathbf{V_1}, \mathbf{V_2}$ to define any point within the triangle. However, this is only true when barycentric coordinates are used. Given an origin $\mathbf{O}$, a triangle with the aforementioned vertices, and any point in space $\mathbf{P}$, the barycentric coordinates $(\lambda_1, \lambda_2, \lambda_3)$ are defined by the following property:

$$(\lambda_1 + \lambda_2 + \lambda_3) \cdot \mathbf{OP} = \lambda_1 \cdot \mathbf{OV_0} + \lambda_2 \cdot \mathbf{OV_1} + \lambda_3 \cdot \mathbf{OV_3}. \tag{2.2}$$

The barycentric coordinates are unique up to a scaling, which implies that there always exists a real number for which the sum of λ_i is normalized to 1. As a corollary, it is obvious that there are two degrees of freedom, stemming from those coordinates, therefore, we rewrite the previous equation for any point $\mathbf{T}(\lambda_1, \lambda_2)$ in the following way

$$\mathbf{T}(\lambda_1, \lambda_2) = (1 - \lambda_1 - \lambda_2) \cdot \mathbf{V_0} + \lambda_1 \cdot \mathbf{V_1} + \lambda_2 \cdot \mathbf{V_2}. \tag{2.3}$$

Therefore the intersection can be simply stated as

$$\mathbf{R}(t) = \mathbf{T}(\lambda_1, \lambda_2) \iff \begin{pmatrix} -\mathbf{D}, & \mathbf{V_1} - \mathbf{V_2}, & \mathbf{V_2} - \mathbf{V_0} \end{pmatrix} \cdot \begin{pmatrix} t \\ \lambda_1 \\ \lambda_2 \end{pmatrix} = \mathbf{O} - \mathbf{V_0} \tag{2.4}$$

The solution of this linear system of equations for $(t, \lambda_1, \lambda_2)$ is solved using Cramer's rule

$$\begin{pmatrix} t \\ \lambda_1 \\ \lambda_2 \end{pmatrix} = \frac{1}{(\mathbf{D} \times \mathbf{V_1} - \mathbf{V_0}) \cdot (\mathbf{V_2} - \mathbf{V_0})} \begin{pmatrix} ((\mathbf{O} - \mathbf{V_0}) \times (\mathbf{V_1} - \mathbf{V_0})) \cdot (\mathbf{V_2} - \mathbf{V_0}) \\ (\mathbf{D} \times (\mathbf{V_2} - \mathbf{V_0})) \cdot (\mathbf{O} - \mathbf{V_0}) \\ ((\mathbf{O} - \mathbf{V_0}) \times (\mathbf{V_1} - \mathbf{V_0})) \cdot \mathbf{D} \end{pmatrix} \tag{2.5}$$

with the only stored variables being the vertices.

2.3 Triangle Search Optimization - Octree

At this stage, it would seem as if there is nothing left to implement, since the particle tracing part and the wall collision checking part are all complete. However, it is important to reiterate, that at each timestep, for each particle, the program has to perform collision checks. Even though that algorithm is fast without the need for large memory allocation, the fact of the matter is, there is a an immense number of triangles to go through, which will render the tool very slow, if the only consideration with regards to searching is brute force.

The problem at hand is the reduction of the number of triangles that are going to undergo the collision check at a given timestep. The wall which contains all the triangles is in the form of a triangular mesh, as mentioned before. This can be seen as a structure similar to a point cloud in 3D space. Intuition states that the triangles that ought to be checked would have to be the ones that are in close proximity relative to the last two position vectors of each particle. Therefore, the 3D space containing the mesh should be partitioned accordingly, in order to achieve an exponential reduction of the number of triangles checked.

The octree data structure is a fitting example of an algorithm which is mostly used for partitioning of 3D space, as the extension of the quadtree in 3 dimensions [16] (figure 4). It is defined as a hierarchical data structure recursively partitions the 3D space into 8 congruent disjoint subsets called octants until a certain maximum depth, i.e., number of iterated recursions has been executed.

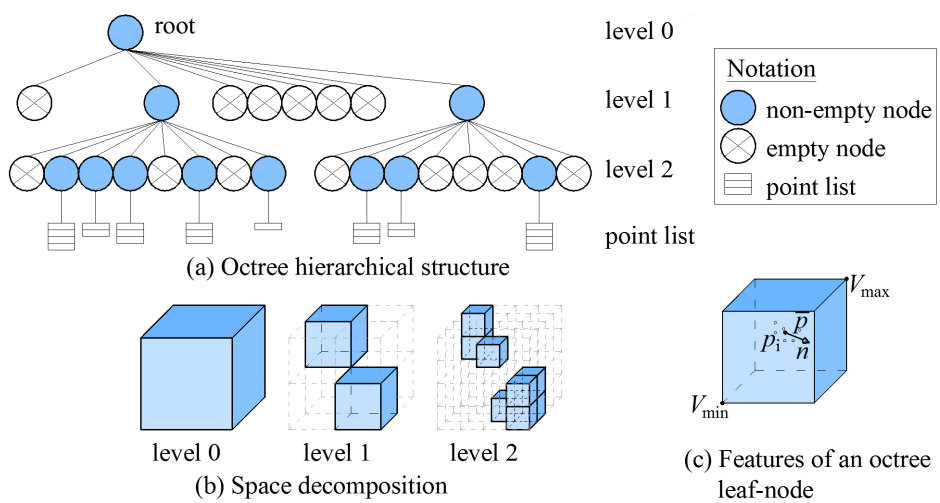


Figure 4: The Octree decomposition scheme, adapted from [17]

The major advantage of this structure is that it can work on any arbitrary 3D surface until the recursion is complete [18]. Additionally it is versatile enough to be altered in such a way that instead of reading single points, triangles are processed instead. The algorithm to construct the octree structure is simple for each iteration:

1. Read all the triangles that constitute the mesh for a given boundary volume.

2. Partition the 3D space into 8 octants (nodes).
3. For every node, first make sure its depth is less than the user defined maximum, otherwise stop.
4. Find how many triangles are contained within it. If there are 0 contained triangles, no need to partition this node further, otherwise repeat the process. A triangle is contained within a node if it's bounding box overlaps the node's volume.

The crucial part of the algorithm is the one that is responsible for the search within the octree after it has been created. User given inputs, in this case, the two position arrays of a runaway electron, are fed into the routine that determines which nodes they belong in. The triangles contained within this node are the ones that will be checked by the ray tracing routine instead of the whole mesh. The algorithmic complexity for the search Octree is $\mathcal{O}(\log_2 n)$ whereas the space complexity is $\mathcal{O}(k \log_2 n)$, where k is the number of triangles and n the number of nodes [19]. The brute force complexity is $\mathcal{O}(k^2)$, which implies that for the $\sim 5 \cdot 10^5$ triangles that constitute the mesh we will be using, for each one marker, the brute force code can be at least k or 500,000 times slower.

3 Results

At this point, it would be useful to recapitulate on the way this tool is supposed to function. Firstly, information is being assessed by importing a file from the ITER disruption simulation at the end of the runaway current plateau, and through the JOREK post-processing tools we obtain the runaway electron density. This density is then fed into a Python script which generates the runaway electron markers. Those markers are pipelined to the relativistic guiding center tracer which simulates their kinetics and dynamics for a user-given amount of time. Before the simulation takes off, the octree routines initialize the structure (with a user provided maximum depth) and then for every marker, at each timestep, the octree search routine finds which node the marker's initial and final positions belong to, and then checks for collisions against the triangles contained only within those nodes. Whenever there is an intersection of the particle ray with any triangle, the particle is considered as lost and will be not be traced further. Finally all the relevant information is written into an output file for post-processing purposes and analysis.

As a last sidenote, in order to match the JOREK wall grid and the ITER wall model, the latter had to be shifted 20 cm downward. Also, all of the simulations were run using the Marconi supercomputer operated by CINECA in Italy, on the partitions designated for fusion research.

3.1 Octree Convergence

The first test runs were dedicated to benchmark the time required to simulate all of the markers' motion while utilising octree, for a variating maximum depth. In other words, a base case was set up in order to validate or falsify the "ansatz" that the use of octree reduced exponentially the CPU time required for the simulations.

For the base case, the marker number was set to 10^4, with a kinetic energy of 1 MeV and a simulation time of 10 μs. The results are displayed on figure 5.

The reason behind the limited selection of values for the maximum depth stems from the suspicion that the computational time required for shallow octrees rose exponentially. This was confirmed by the fit performed on the data points collected from this "benchmarking test". For a maximum depth of 3 the CPU time is already on the 350,000 seconds ballpark. The fit performed on the data points is an asymptotic exponential of the form

$$y = A + be^{-\frac{t}{t_0}} \tag{3.1}$$

The values of the parameters defined in the fitted expression are

$$A = 19980 \text{ s} \tag{3.2}$$
$$b = 7.62 \cdot 10^7 \text{ s} \tag{3.3}$$
$$t_0 = 0.39 \tag{3.4}$$

All with relative errors of less than 3%. From this curve it was inferred that for the depths of 2 and 1 the CPU time would rise to $3 \cdot 10^6$ seconds and $48 \cdot 10^6$ seconds, respectively, which are equivalent to 16 and 100 hours of runtime on Marconi when using 4 nodes or 192 threads.

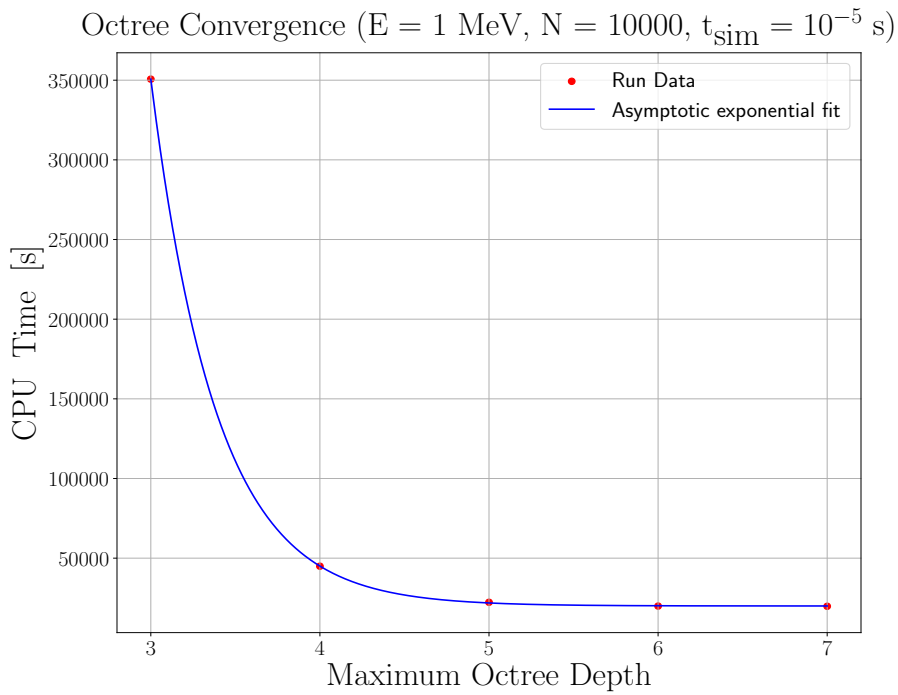


Figure 5: The amount of CPU time required to simulate the markers, each time for a different maximum depth of the octree structure. The blue curve is the asymptotic exponential fit used on the run data.

The most significant takeaway from this convergence study, is that, indeed the octree structure speeds up and facilitates the collision checks immensely along with the observation that after a maximum depth of 5 there is very little variation in the CPU time i.e. the simulation time converges near the value of the parameter A from the fit at around 20,000 seconds.

3.2 Number of Markers Scan

Before production applications, the convergence of results with the number of markers is tested.

The results, as can be seen on figure 8, meaning that for the short and intermediate simulation times, the lost particles fraction remains under 1% with no particular indication of a certain trend arising while varying the number of markers.

When one simulates for a longer period of time, this fraction sees a sharp shift to the 90% range, while remaining almost constant along the increase in the number of markers. The

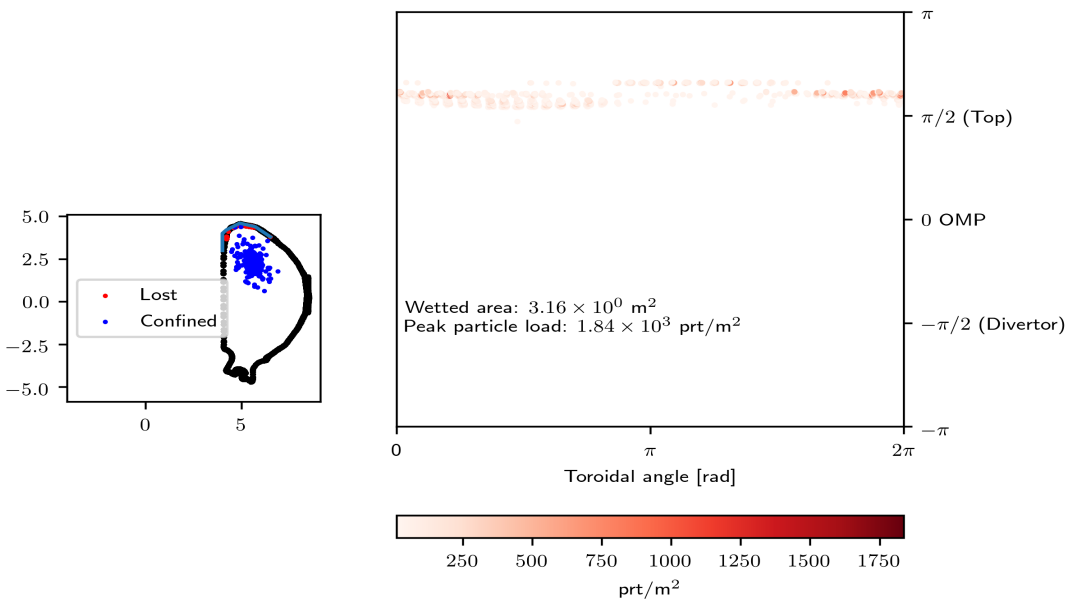


Figure 6: The (R,Z) plot on the left displays which markers have been registered as lost or confined. The blue curve is used in postprocessing to measure how far along the wall losses span. The (ϕ, θ) plot shows the surface density of the hits along the torus. This example is the result of a simulation of 10,000 markers with a kinetic energy of 100 MeV each.

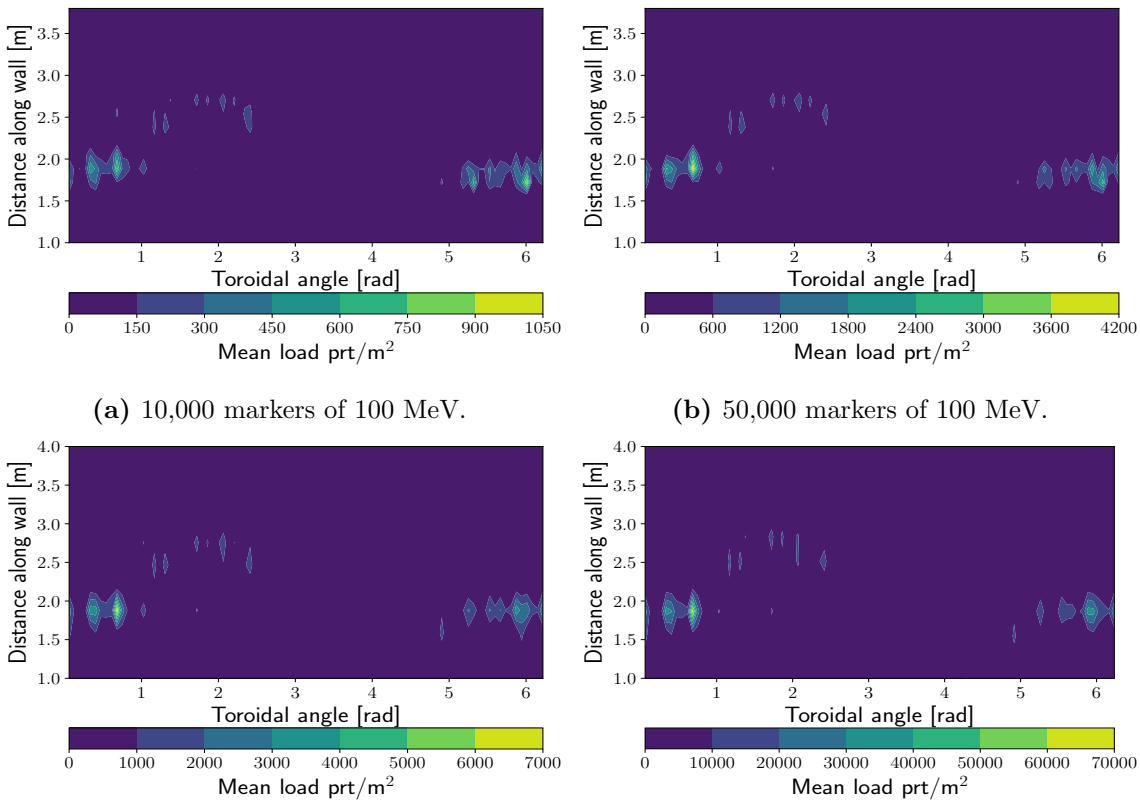
small fluctuations that are barely noticeable, stem from the use of Monte Carlo methods for all simulation times.

It would be interesting and appropriate to investigate how the distribution of losses along the wall changes with the variation of the number of markers.

From figure 6 one can directly extract information about the loss distribution pattern. The darker points represent high loads while the fainter ones correspond to low loads (the prt/m^2 units refer to test particles or markers per unit surface and not actual particles). The striking part about their distribution is that the former are contained within a specific range of toroidal angles, whereas in the complementary range of toroidal angles there are little to no losses. This is an indication that the $n = 1$ MHD mode in the field structure strongly affects losses.

The loss patterns on the wall can be better studied by looking how the hits are distributed along the aforementioned curve from the (R, Z) plot. The (R, Z) is similar among all cases, therefore we will just compare the wall patterns.

The patterns arising in figure 7 are more interesting. An intermittent type of distribution (toroidally) is seemingly in effect. In other words, there are some areas which approximately seem to span $\pi/3$ rad on either side of the 0 rad toroidal angle point, where amorphous



(a) 10,000 markers of 100 MeV.

(b) 50,000 markers of 100 MeV.

(c) 100,000 markers of 100 MeV.

(d) 1,000,000 markers of 100 MeV.

Figure 7: The spatial distribution of the hits when projected along the curve of the wetted area of the wall, for a different number of markers all of which had a kinetic energy of 100 MeV.

structures form in which the concentration of hits is 2-3 orders of magnitude larger than in surrounding areas. Lastly, there exist some very small areas with increased hit concentration along an almost fixed distance along the wall.

From figure 7, one can quickly deduce that there is not a lot of variation to the form of the spatial distribution itself. With increasing number of markers the medium concentration spots that are situated at the bottom of subfigures 7a and 7b are becoming more pronounced. This resulted in them becoming large enough that they are no longer practically negligible.

Convergence can be ascertained by determining whether primarily the peak particle load and less so the wetted area, converge to a certain value. From the simulation data in figure 9 it is clear that the peak particle load is on the way to converge to some certain value, indicative of which is the load for 100,000 markers, whereas the wetted area is not converging as fast. As a matter of fact, the wetted area is not as significant because the loads are peaked. It doesn't matter where few electrons end up as long as the wall loads are tolerable.

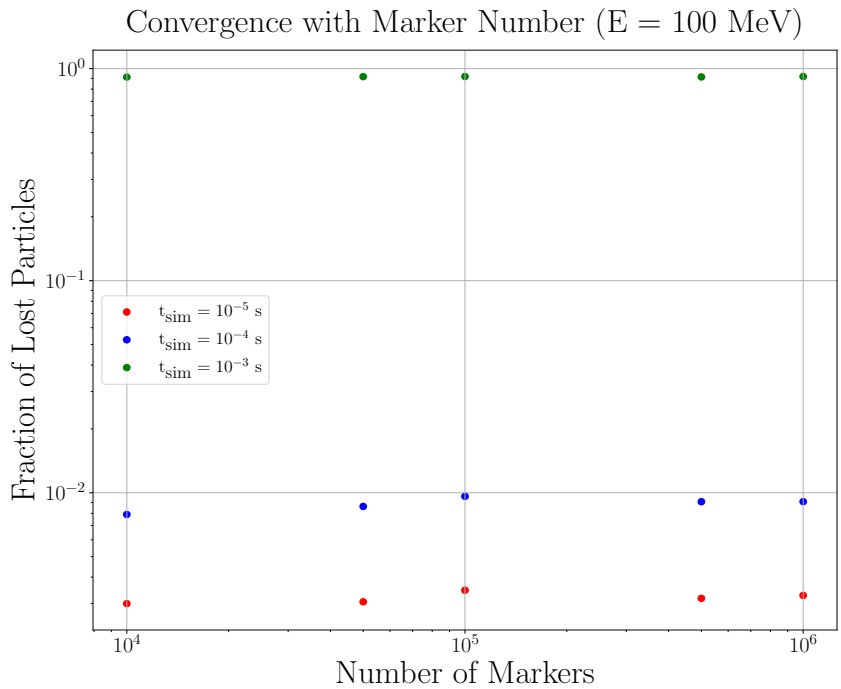
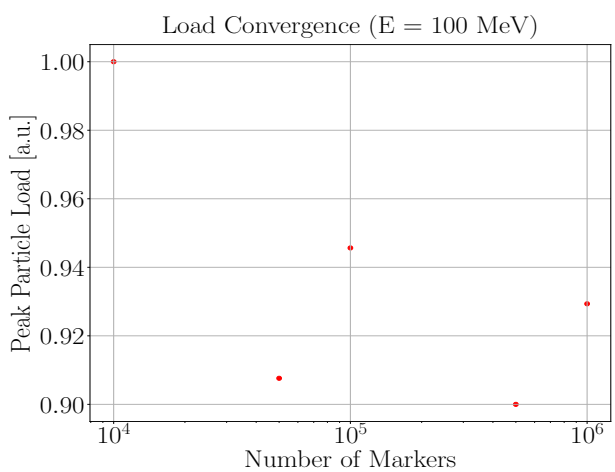
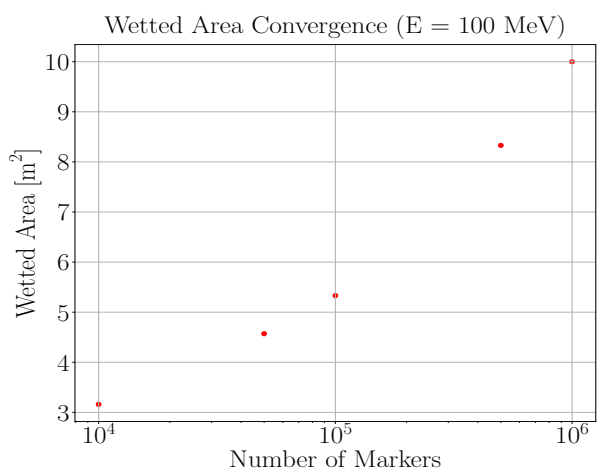


Figure 8: The fraction of lost particles during a complete simulation for a runtime of 0.01, 0.1 and 1 ms, for a default energy of 100 MeV.



(a) The normalized peak particle load.



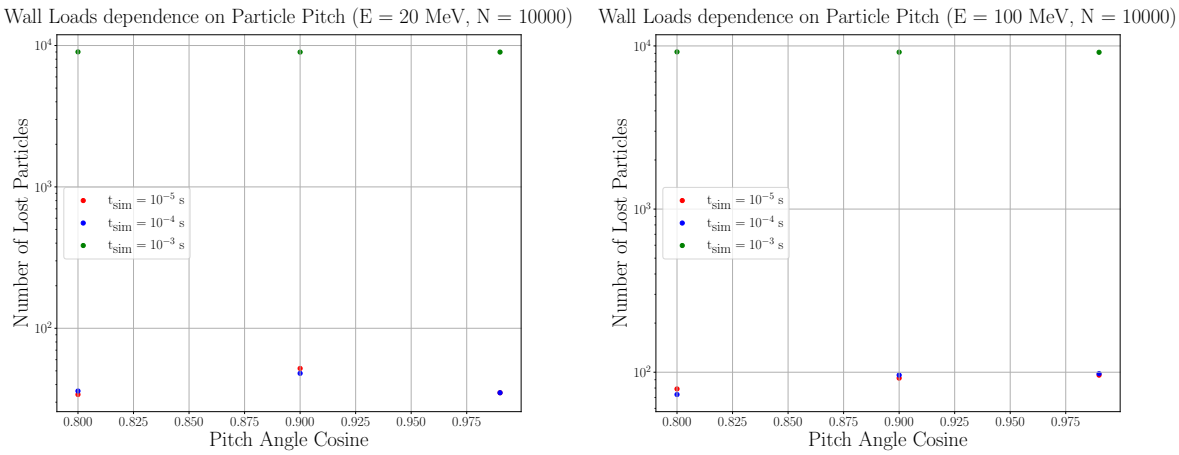
(b) The wetted area.

Figure 9: The normalized peak particle load and wetted area for an increasing number of markers.

3.3 Pitch Angle Scan

Given the fact that the convergence test showed that the pattern of the highest loads can be trusted, we are therefore now able to perform scans across electron energy and pitch to

see how those affect the wall load distribution. What we refer to as pitch in this work is the pitch angle's cosine, i.e., the ratio of the parallel velocity component and the total velocity of the particle. Pitch could potentially provide a measure of how fast particles are lost, since transport is due to particles travelling across the stochastic field lines, i.e. a higher parallel velocity component would imply faster losses. Also since the plasma is moving throughout the simulation, the pitch might affect the wall pattern as particles are lost.



(a) The 20 MeV pitch scan.

(b) The 100 MeV pitch scan.

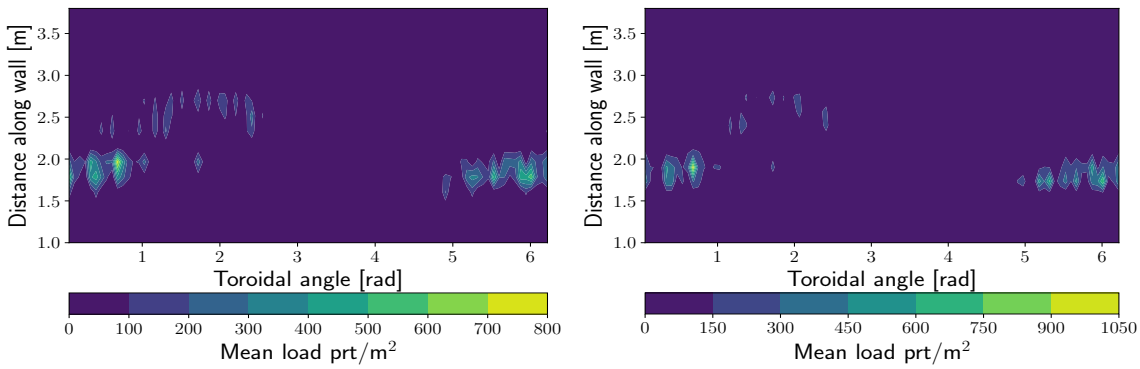
Figure 10: The number of lost particles for different values of the pitch, for 10000 runaway electrons of 20 and 100 MeV. Each color represents the simulation time used in each case, spanning 2 orders of magnitude.

It is apparent for both cases that there is a significant difference in the number of particle that are lost as simulation time goes on. For the relatively shorter intervals of simulated time, i.e., 0.01 and 0.1 ms, there is no clear trend in the behavior of the number of lost particles as the pitch cosine increases. On the contrary, for either value of the kinetic energy, the fraction of lost particles is always below 1% and fluctuating within a very small interval, when one varies either the pitch angle or the simulation time. In fact, since the figures are so low, one may disregard those variations as just Monte Carlo noise.

On the other hand there is a much more consistent behavior once the simulation time is increased to 1 ms. Firstly, it is clear that the amount of particles lost has risen drastically, from less than 1% to about 90%, and that seems to be the case for either value of the kinetic energy assigned to the runaways.

In figure 11 the two distributions display subtle differences. The main being that the structures that arose due to the density of hits, i.e., the mean peak load at each area, are more diffuse for a lower pitch and more distinct and separate as it approaches unity.

One may speculate as to why this is, one probable reason could be that for a lower pitch cosine, the particle has a larger perpendicular velocity component, which results in this more diffuse situation. However, this requires a study of incidence angle of the particles which is out of the scope of this report, therefore this discussion will not be continued further.



(a) $N = 10^4$, $E_{\text{kin}} = 100$ MeV, pitch $= 0.8$. **(b)** $N = 10^4$, $E_{\text{kin}} = 100$ MeV, pitch $= 0.99$.

Figure 11: The spatial distribution of the hits along the curve of the wetted area of the wall, for a varying value of the pitch.

3.4 Energy Scan

As was the approach in the previous sections, the first investigation when it comes to changing the energy of the markers, revolves around testing the amount of particles that get lost for a given simulation time.

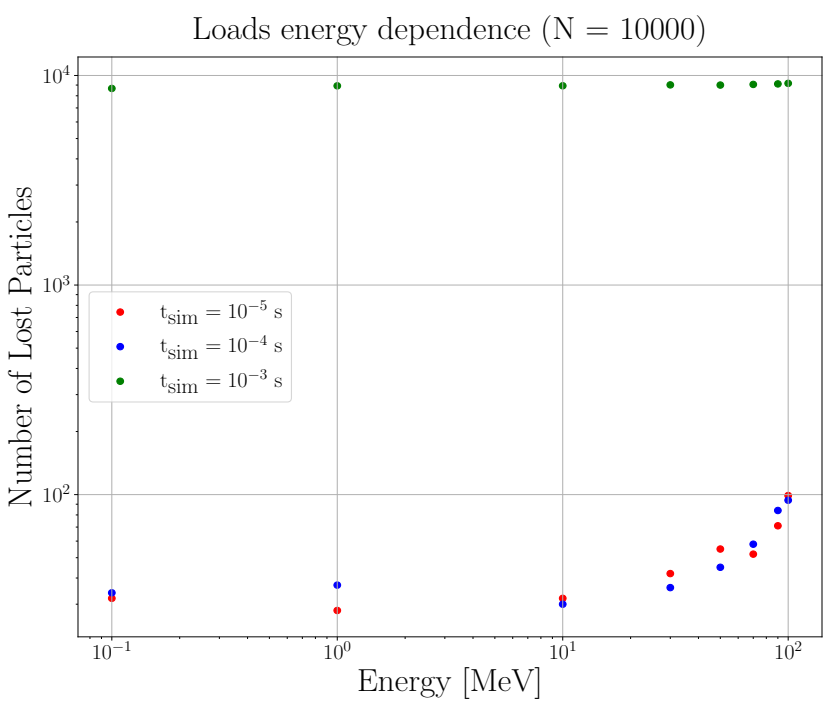


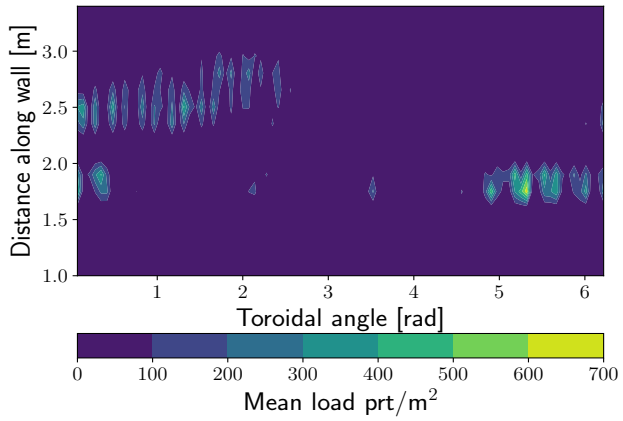
Figure 12: The total number of lost particles as a function of the energy of the markers for varying simulation times. The total number of markers was 10,000.

Even though, one may dismiss this dependence as being identical to the previous ones, such

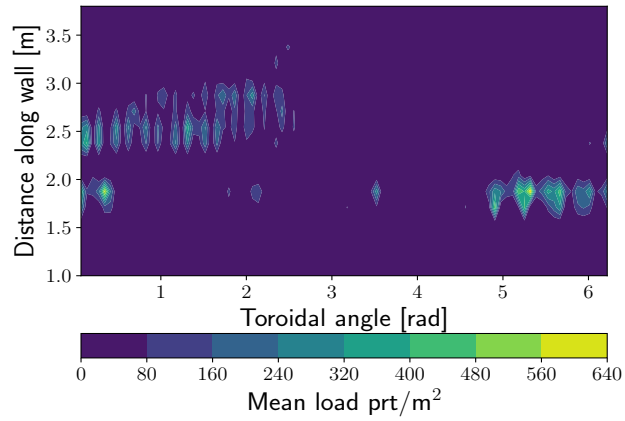
as the one in figures 3 and 10 there is a very interesting detail present here in figure 12. In particular, for the intermediate and short simulation times, there is a sharp increase in lost particles after the threshold of 10 MeV. This increase in lost particles ends up barely reaching 1%, nevertheless it remains a display of what the physics at play here. We conjecture that a possible mechanism behind this observation is that, the more energetic particles have a larger orbit shift which renders them more susceptible to come in contact with the wall faster. That is, even though the ultimate fate of 90% of the particles is to become lost after 1 ms, regardless of the energy of the markers, the higher energetic particles escape first up to a certain point.

Figure 13 displays the different distributions one obtains after running a scan for the kinetic energy of the markers. Upon initial inspection it is immediately noticed that the mean peak load remains within the same order of magnitude across the energy spectrum, with a maximum deviation of 50% on the high end of the colorbar scale.

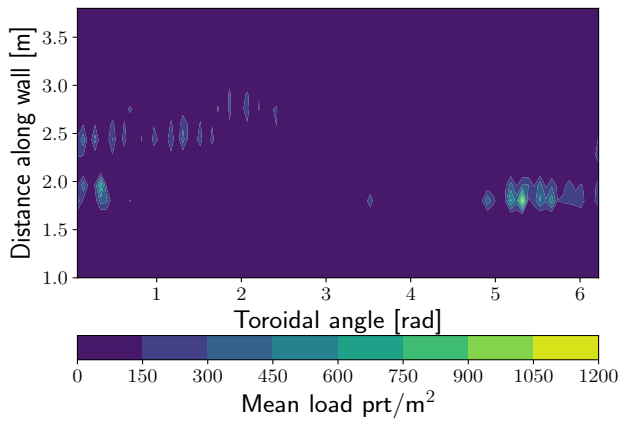
Further examination of figures 13a to 13f reveals one peculiar pattern emerging. The long and narrow contours that were in very close proximity to each other for the distributions at the lower end of the scanned spectrum, seem to dissipate and disappear as the energy increases. Additionally, the mean peak load within those contours decreases as well, therefore making them appear as fainter on the plot. This decrease is apparently compensated with an increase of the mean peak load at the brighter areas. This is interesting especially considering that the peak loads are not that much affected.



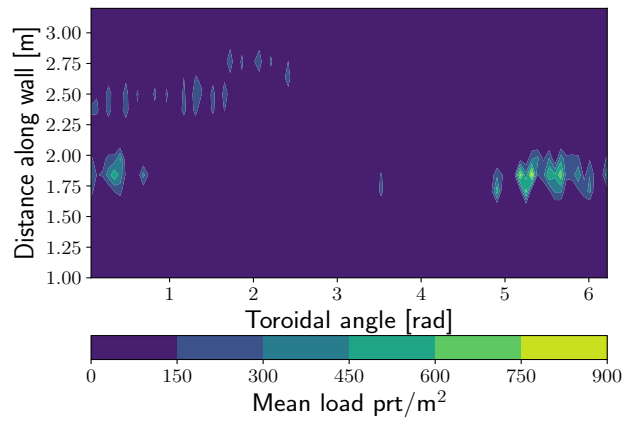
(a) 0.1 MeV.



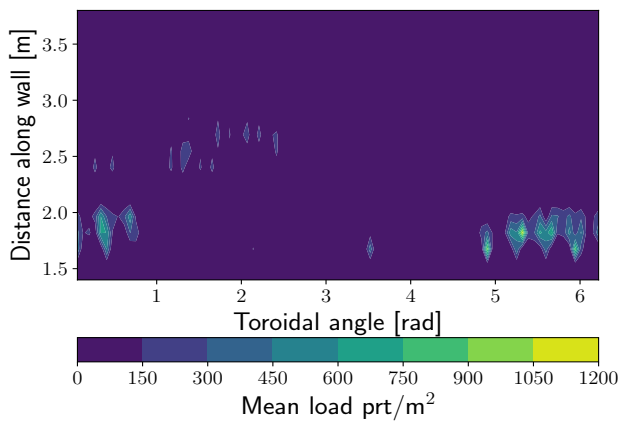
(b) 1 MeV.



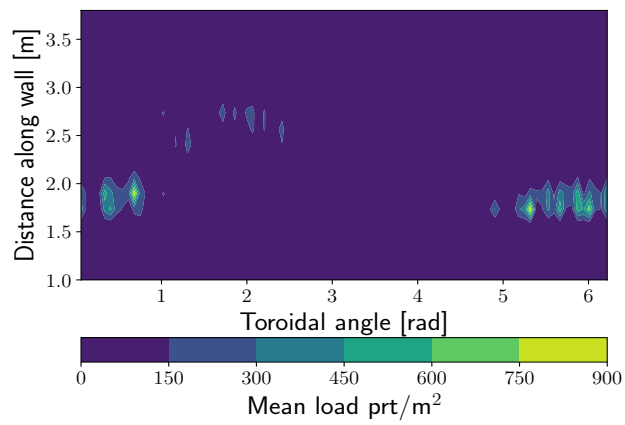
(c) 10 MeV.



(d) 30 MeV.



(e) 50 MeV.



(f) 90 MeV.

Figure 13: The spatial distribution of the hits along the curve of the wetted area of the wall, for a varying value of the kinetic energy of the markers.

3.5 Preliminary Heat Loads Results

The initial goal set for this report was to obtain the spatial distribution of the runaways on the first wall and also perform further analysis whether this implied scanning certain parameters to see whether their effect is significant or not, or if there is an emergent pattern.

In the introduction it was explicitly mentioned that although we may acquire the spatial distributions on the wall, that would not trivially imply that automatically one possesses the energy deposition distributions of the runaways on the wall. However, we are in a position to present some preliminary results which feature both. The results correspond to 100,000 markers with a kinetic energy of 50 MeV.

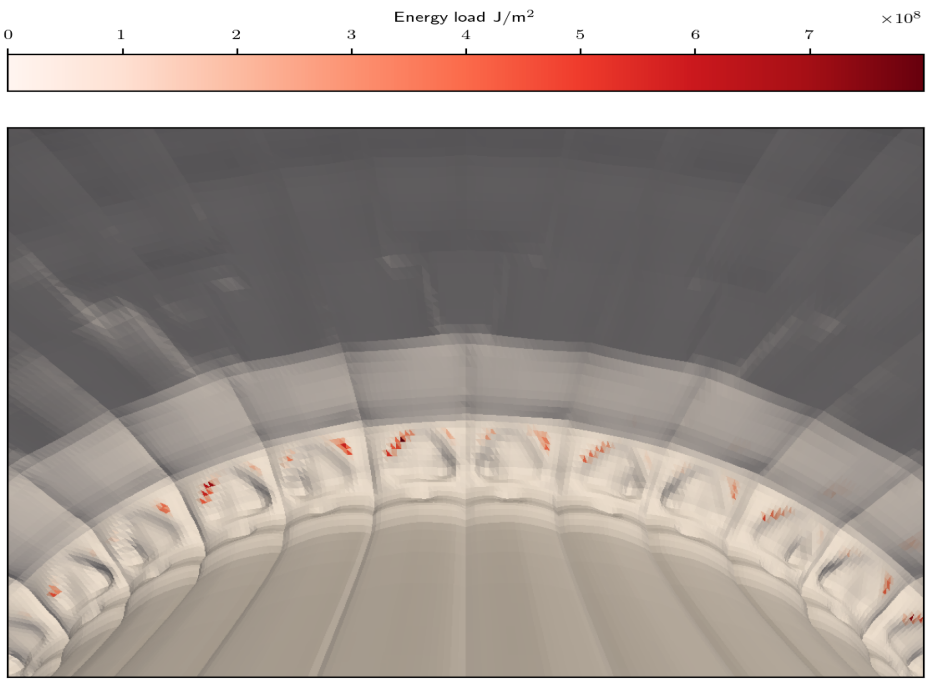


Figure 14: The 3D representation of the model of the wall with the energy load from the losses is being indicated by the coloration of the cluster of triangles. Note that the wall model used for this demonstration does not exactly reflect the ITER wall structure such that our results do not provide quantitative predictions at this stage.

The scale in figure 14 reveals a terrifying scale for the peak energy load, reaching up to $\sim$800 MJ/m^2. However, this not a direct cause for concern, since there has to be some context provided. The 50 MeV for the runaway markers is already on the high end of the runaway energy spectrum expected to be expected in ITER. From figure 16 one can tell that the previously mentioned $n = 1$ MHD mode contributes to the heat load. Finally, there is a certain sort of fine structure present in figure 14 i.e. the energy loads are concentrated within the edges of the valley-like structures in the wall, while wall structures used in this demonstration are not fully realistic.

Nevertheless, it should be noted that even though those results are on the higher end and include significant uncertainties , the results emphasize how critical RE losses are for ITER and how important qualitative predictions based on this diagnostic tool will be. For ITER, serious damage can occur above a threshold of about 10 MJ/m^2 [20, 21].

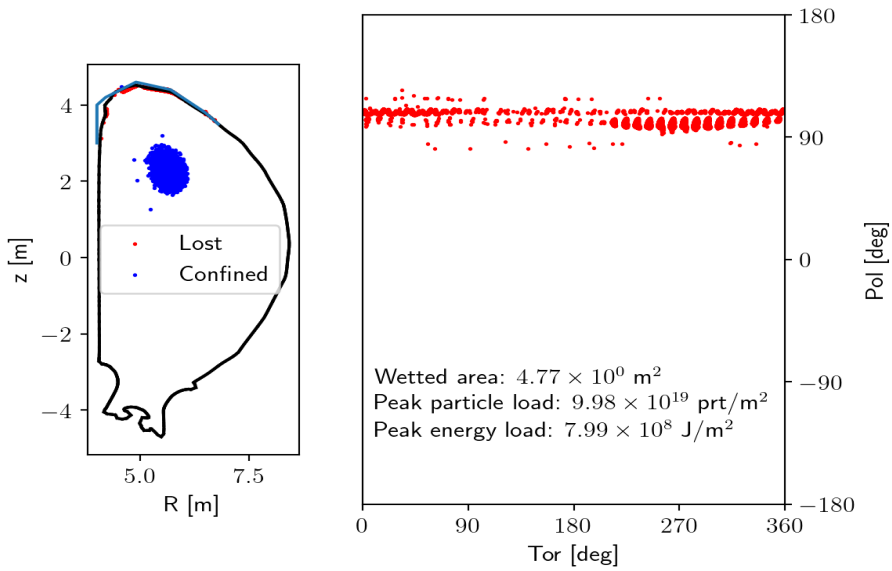


Figure 15: The (R,Z) plot shows where the particles were deposited and the (ϕ, θ) plot depicts the particles deposition around the torus.

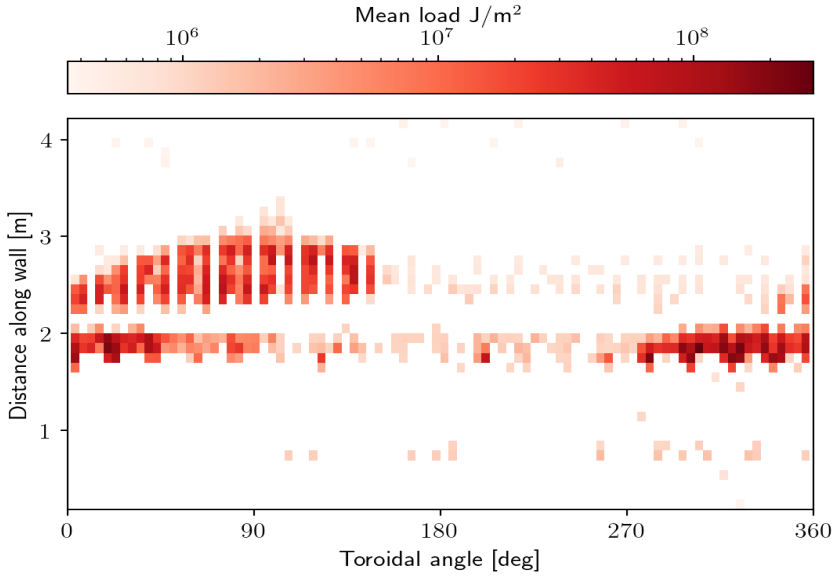


Figure 16: The heat flux distribution projected onto the blue curve from figure 15, across the torus.

4 Summary & Future Work

Runaway electrons are one of the main issues that fusion currently faces as a scientific field because of their perilous potential to attain such high energies that the materials constituting the first wall will not be able to handle the resulting heat loads.

This is why in this project, a virtual diagnostic tool is built, tested and demonstrated for obtaining the spatial distribution of runaways lost during the disruption. This program consists of two main components, the ray tracing routine that performs the wall collision check for every particle and the octree construction routine, which performs the segmentation of the 3D space into octants to facilitate the selection of the triangles required to be checked by the former routine for wall collisions and drastically reduce computational costs.

The wall collision check routine works by utilising the Möller - Trumbore algorithm which involves translating and transforming the line segment and triangle, in question for intersection. This particular algorithm was selected due to its speed and minimum storage requirements.

The wall collision check and the octree structure were integrated into a relativistic guiding center tracer. Then, data from ITER disruption simulations was used to simulate the runaway dynamics, at the point were the runaway current plateau ended by a burst of MHD modes.

The first test that was conducted for the report was the convergence of the octree construction and search routines. As expected, the time that was needed to simulate the same amount of markers for octrees of larger depth decreased exponentially. This provided us with the green light to proceed to benchmarks and applications.

Next, the number of simulated markers was chosen as the first parameter of the simulations to scan, since there is another issue related to convergence arising here. The simulation time tests showed homogeneous fractions of simulated lost particles across the different numbers of markers used. The peak particle loads also seemed to converge, which is the most important parameter that needed to exhibit this behaviour. After those tests, the first spatial distributions were acquired and analyzed.

The scan in the pitch angle cosine was to follow. First we validated that no matter the pitch, the same amount of particles would be lost for each simulation duration. This was true for both energies that were studied. The spatial distribution across the torus exhibited some reduction in the mean peak load in certain areas as the pitch angle cosine increased, with a compensating effect on the rest of the already high load regions.

The subsequent energy scan revealed that energetic particles get in contact with the wall more quickly and that the mean peak load across the torus remains fairly stable with increasing energy, with the losses concentrating more on the high-load regions.

The final result presented here was a preliminary result for 50 MeV electrons and their corresponding energy distribution. There is an expected reduction in the heat load due to statistics and the fact that 50 MeV runaways are on the high end of what is expected for ITER. Nevertheless, albeit the way the heat loads were calculated is crude as an approximation, the wall materials would undergo massive damage since the heat flux surpasses the

safety thresholds.

This project constitutes a first and important step to qualitative ITER predictions. There is still room for a lot of progress, statistics-wise and approximation-wise, especially when it comes to the estimation of the runaways' energy. A way this might be achieved is through marker tracing during the plateau phase as well as the inclusion of the E-field and both collisional and radiation losses, from which the maximum energy of a runaway could be deduced. The tool can then be applied also to other ITER simulations.

The combination of the aforementioned future work could result in an estimate for the maximum tolerable runaway current, which can be beneficial to know for the development or optimization of future mitigation or prevention techniques.

References

1. Breizman, B. N., Aleynikov, P., Hollmann, E. M. & Lehnen, M. Physics of runaway electrons in tokamaks. *Nuclear Fusion* **59,** 083001. ISSN: 0029-5515. https://iopscience.iop.org/article/10.1088/1741-4326/ab1822 (June 2019).
2. Lehnen, M. *et al.* Disruptions in ITER and strategies for their control and mitigation. *Journal of Nuclear Materials* **463,** 39–48. ISSN: 0022-3115. https://doi.org/10.1016/j.jnucmat.2014.10.075 (Aug. 2015).
3. Boozer, A. H. Theory of tokamak disruptions. *Physics of Plasmas* **19.** ISSN: 1070664X. https://doi.org/10.1063/1.3703327 (May 2012).
4. Dreicer, H. Electron and Ion Runaway in a Fully Ionized Gas. I. *Physical Review* **115,** 238. ISSN: 0031899X. https://journals.aps.org/pr/abstract/10.1103/PhysRev.115.238 (July 1959).
5. Dreicer, H. Electron and ion runaway in a fully ionized gas. II. *Physical Review* **117,** 329–342. ISSN: 0031899X. https://doi.org/10.1103/PhysRev.117.329 (1960).
6. Helander, P., Smith, H., Fülöp, T. & Eriksson, L. G. Electron kinetics in a cooling plasma. *Physics of Plasmas* **11,** 5704. ISSN: 1070-664X. https://aip.scitation.org/doi/abs/10.1063/1.1812759 (Nov. 2004).
7. Chiu, S. C., Rosenbluth, M. N., Harvey, R. W. & Chan, V. S. Fokker-Planck simulations of knock-on electron runaway avalanche and bursts in tokamaks. *Nuclear Fusion* **38,** 1711. ISSN: 0029-5515. https://iopscience.iop.org/article/10.1088/0029-5515/38/11/309 (Nov. 1998).
8. Putvinski, S. *et al.* Halo current, runaway electrons and disruption mitigation in ITER. *Plasma Physics and Controlled Fusion* **39,** B157. ISSN: 0741-3335. https://iopscience.iop.org/article/10.1088/0741-3335/39/12B/013 (Dec. 1997).
9. *"Multiplication" of accelerated electrons in a tokamak* https://www.osti.gov/biblio/5974839.
10. Rosenbluth, M. N. & Putvinski, S. V. *Theory for avalanche of runaway electrons in tokamaks* tech. rep. (1997), 1355. http://iopscience.iop.org/0029-5515/37/10/I03.
11. Papp, G., Drevlak, M., Fülöp, T. & Pokol, G. I. Spatial distribution of ITER runaway electron wall loads in the presence of 3D magnetic perturbations. *39th EPS Conference & 16th Int. Congress on Plasma Physics.* https://pure.mpg.de/rest/items/item_2146142/component/file_3320592/content.
12. Bandaru, V. *et al. private communication*
13. Hoelzl, M. *et al.* The JOREK non-linear extended MHD code and applications to large-scale instabilities and their control in magnetically confined fusion plasmas. *Nuclear Fusion* **61,** 065001. ISSN: 0029-5515. https://iopscience.iop.org/article/10.1088/1741-4326/abf99f (May 2021).
14. Bandaru, V., Hoelzl, M., Artola, F. J., Papp, G. & Huijsmans, G. T. Simulating the non-linear interaction of relativistic electrons and tokamak plasma instabilities: Implementation and validation of a fluid model. *Physical Review E* **99,** 063317. ISSN: 24700053. https://journals.aps.org/pre/abstract/10.1103/PhysRevE.99.063317 (6 June 2019).

15. Möller, T. & Trumbore, B. Fast, Minimum Storage Ray-Triangle Intersection. *Journal of Graphics Tools* **2,** 21–28. ISSN: 1086-7651. https://www.tandfonline.com/doi/abs/10.1080/10867651.1997.10487468 (Jan. 2012).
16. Samet, H. An Overview of Quadtrees, Octrees, and Related Hierarchical Data Structures. *Theoretical Foundations of Computer Graphics and CAD,* 51–68. https://link.springer.com/chapter/10.1007/978-3-642-83539-1_2 (1988).
17. Vo, A. V., Truong-Hong, L., Laefer, D. F. & Bertolotto, M. Octree-based region growing for point cloud segmentation. *ISPRS Journal of Photogrammetry and Remote Sensing* **104,** 88–100. ISSN: 09242716. https://www.researchgate.net/publication/274645446_Octree-based_region_growing_for_point_cloud_segmentation (June 2015).
18. Carlbom, I., Chakravarty, I. & Vanderschel, D. A Hierarchical Data Structure for Representing the Spatial Decomposition of 3D Objects. *Frontiers in Computer Graphics,* 2–12. https://link.springer.com/chapter/10.1007/978-4-431-68025-3_1 (1985).
19. Castro, R., Lewiner, T., Hélio Lopes, Tavares, G. & Bordignon, A. Statistical optimization of octree searches. *Computer Graphics Forum* **27,** 1557–1566. ISSN: 1467-8659. http://dx.doi.org/10.1111/j.1467-8659.2007.01104.x (2008).
20. Bazylev, B., Arnoux, G., Fundamenski, W., Igitkhanov, Y. & Lehnen, M. Modeling of runaway electron beams for JET and ITER. *Journal of Nuclear Materials* **415.** Proceedings of the 19th International Conference on Plasma-Surface Interactions in Controlled Fusion, S841–S844. ISSN: 0022-3115. https://www.sciencedirect.com/science/article/pii/S0022311510007944 (2011).
21. Chen, L. *et al.* Modeling runaway electron induced damage to ITER plasma-facing components. *5th Asia-Pacific Conference on Plasma Physics.* http://aappsdpp.org/DPP2021/MF-2/5155.pdf (2021).